FA-2 CASE STUDY ASSIGNMENT
DBMS - NoSQL

Project: Employee Performance Evaluation

1. MongoDB Collection Design - Equivalent to 16 SQL Tables.

The Employee Performance Evaluation System relational schema consists of 16 normalized SQL tables. For MongoDB implementation, these tables are denormalized into 4 major collections. Data that is logically accessed together is embedded into the same document, which reduces the need for JOIN operations and improves read performance. This document-oriented design is more suitable for flexible and hierarchical data such as reviews, training history, goals, feedback, and rewards.

Collection 1: employees

```
{
 _id: 103,
 name: "Ayush Thakare",
 department: {
   departmentId: 10,
   departmentName: "IT"
 },
 manager: {
   managerId: 201,
   managerName: "Rajesh Patil"
 },
 projects: [
   {
     projectId: 301,
     projectName: "Employee Evaluation Portal"
   }
 ]
}
```

Collection 2: reviews

```
{
 _id: 507,
 employeeId: 103,
 managerId: 201,
 reviewDate: ISODate("2026-03-15T00:00:00Z"),
 rating: 4.4,
 skillScores: [
   {
     skillId: 401,
     skillName: "SQL",
     score: 4.5
   },
   {
     skillId: 402,
     skillName: "Communication",
     score: 4.7
   }
 ],
 goals: [
   {
     goalId: 301,
     description: "Improve SQL optimization"
   }
 ],
 feedback: [
   {
     reviewerId: 202,
     reviewerName: "Sneha Kulkarni",
     comment: "Excellent analytical performance"
   }
 ]
}
```

Collection 3: trainings

```
{
 _id: 801,
 trainingName: "Advanced SQL Workshop",
 skill: {
   skillId: 401,
   skillName: "SQL"
 },
 enrolledEmployees: [
   {
     employeeId: 103,
     employeeName: "Ayush Thakare",
     enrollmentDate: ISODate("2026-03-20T00:00:00Z")
   }
 ],
 certifications: [
   {
     certificationId: 901,
     employeeId: 103,
     issueDate: ISODate("2026-04-10T00:00:00Z")
   }
 ]
}
```

Collection 4: rewards

```
{
 _id: 1001,
 employeeId: 103,
 reviewId: 507,
 bonus: {
   bonusId: 601,
   amount: 15000,
   approvedBy: 201
 },
 promotion: {
   promotionId: 701,
   newRole: "Senior Developer",
   approvedBy: 201
 },
 recognition: {
   recognitionId: 801,
   title: "Employee of the Month",
   recognizedBy: 202
 }
}
```

2. MongoDB Queries - 10 Queries Covering All Required Operations.

Query 1 - CREATE: Insert a New Employee

```
db.employees.insertOne({
  _id: 103,
  name: "Ayush Thakare",
  department: {
    departmentId: 10,
    departmentName: "IT"
  },
  managerId: 201,
  projects: [
    { projectId: 301, projectName: "Employee Evaluation
Portal" }
  ]
});
```

Query 2 - CREATE: Insert a Performance Reviews

```
db.reviews.insertOne({
  _id: 507,
  employeeId: 103,
  managerId: 201,
  reviewDate: ISODate("2026-03-15T00:00:00Z"),
  rating: 4.4,
  skillScores: [
    { skillId: 401, score: 4.5 },
    { skillId: 402, score: 4.7 }
  ],
  goals: [
    { goalId: 301, description: "Improve SQL
optimization" }
  ],
  feedback: [
    { reviewerId: 202, comment: "Excellent analytical
performance" }
  ]
});
```

Query 3 - READ: Find All Employees in the IT Department

```
db.employees.find({
  "department.departmentName": "IT"
});
```

Query 4 - READ: Find employees who are assigned to more than one project

```
db.employees.find({
 $expr: {
   $gt: [{ $size: "$projects" }, 1]
 }
});
```

Query 5 - UPDATE: Add a New Project to an Employee (Array Push)

```
db.employees.updateOne(
 { _id: 103 },
 {
   $push: {
     projects: {
       projectId: 302,
       projectName: "Training Dashboard"
     }
   }
 }
);
```

Query 6 - DELETE: Employees who are not assigned to any project

```
db.employees.deleteMany({
 projects: { $size: 0 }
});
```

Query 7 - DELETE: rewards records where no bonus, promotion, or recognition exists

```
db.rewards.deleteMany({
 bonus: { $exists: false },
 promotion: { $exists: false },
 recognition: { $exists: false }
});
```

Query 8 - AGGREGATE: Find department-wise employee count

```
db.employees.aggregate([
 {
   $group: {
     _id: "$department.departmentName",
     totalEmployees: { $sum: 1 }
   }
 },
 {
   $sort: { totalEmployees: -1 }
 }
]);
```

Query 9 - AGGREGATE: Find all employees whose average review rating is greater than 4.0

```
db.reviews.aggregate([
 {
   $group: {
     _id: "$employeeId",
     averageRating: { $avg: "$rating" },
     totalReviews: { $sum: 1 }
   }
 },
 {
   $match: {
     averageRating: { $gt: 4.0 }
   }
 },
 {
   $sort: { averageRating: -1 }
 }
]);
```

Query 10 - AGGREGATE: Find the highest-rated employee

```
db.reviews.aggregate([
 {
   $group: {
     _id: "$employeeId",
     averageRating: { $avg: "$rating" }
   }
 },
 {
   $sort: { averageRating: -1 }
 },
 {
   $limit: 1
 }
]);
```

Query 11 - INDEXING: Create text index on feedback comments for keyword search

```
db.reviews.createIndex(
 { "feedback.comment": "text" },
 { name: "idx_feedback_text" }
);
```

Query 12 - READ: After creating text index, you should also show one search query

```
db.reviews.find(
 { $text: { $search: "excellent" } },
 { score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } });
```

3. Comparison: SQL vs MongoDB

Feature	Employee Performance Evaluation System SQL (MySQL)	Employee Performance Evaluation System (MongoDB)
Architecture	16 normalized tables (Employee, Department, Manager, Review, Skill, Goal, Feedback, Training, Bonus, Promotion, etc.)	4 core collections (`employees`, `reviews`, `trainings`, `rewards`) with related data embedded in documents
Schema	Rigid relational schema with fixed columns, data types, and constraints. Any change requires `ALTER TABLE`	Flexible / schema-less document model. Documents in the same collection can have slightly different structures
Relationships	Relationships maintained using PRIMARY KEY / FOREIGN KEY constraints	Relationships handled through embedded documents, arrays, and manual references (`employeeId`, `managerId`, etc.)
Normalization	Highly normalized up to 3NF to reduce redundancy and maintain consistency	Denormalized design to reduce JOINs and store logically related data together
ACID Transactions	Strong multi-table ACID transactions supported natively in MySQL using `START TRANSACTION`, `COMMIT`, `ROLLBACK`	MongoDB supports multi-document transactions, but for this project most operations are handled using single-document atomic updates
Querying	Requires JOINs for employee profile, reviews, skill scores, goals, feedback, and rewards data	Most related data can be fetched with a single `find()` because review details, goals, and feedback are embedded

Constraints / Integrity	Strong integrity through NOT NULL, UNIQUE, CHECK, FOREIGN KEY constraints	No strict foreign key enforcement by default; integrity is maintained at the application / design level
Performance (Reads)	Can be slower for complex hierarchical data because multiple tables must be joined	Faster for document-centric reads, such as fetching full employee review history in one query
Performance (Writes)	Efficient for normalized writes because updates are targeted to specific tables	Can involve data redundancy if embedded data needs to be updated in multiple documents
Scalability	Typically vertical scaling (larger server, optimized indexing)	Better suited for horizontal scaling / sharding in large-scale distributed systems
Consistency	Strong consistency due to normalization, constraints, and transaction isolation	Flexible consistency; easier for rapid reads, but consistency must be handled carefully in denormalized data
Use Case Fit	Best for performance evaluation workflows requiring strict integrity: reviews, promotions, bonuses, approvals	Best for analytics, dashboard views, employee profile summaries, review history, and search-heavy features

4. Conclusion:

In conclusion, in the second phase of our Employee Performance Evaluation System, we extended the project by applying transaction management concepts and NoSQL implementation. We identified important transactions, analyzed schedules for conflict and view serializability, and demonstrated ACID properties along with commit, rollback, and deadlock handling. Further, we redesigned the same system in MongoDB by converting the normalized SQL schema into denormalized collections and implemented MongoDB queries covering CRUD, aggregation, indexing, and search. This phase helped us understand both reliable transaction processing in relational databases and flexible document-based operations in NoSQL systems.

GitHub Repositories Link:

https://github.com/sarang-wasamwar/Employee-Performance-Evaluation-System/